

Of course. Here is a concise but detailed architectural overview of the Astoria Open application, suitable for your demo session.

Astoria Open: System Architecture Overview

Astoria Open is an advanced Natural Language to SQL (NL2SQL) application designed to answer questions about a maritime database. It uses a multi-layered approach, combining fast rule-based systems with powerful Large Language Models (LLMs) to provide accurate, context-aware, and credible narrative answers.

Core Components

- **Web Server (Flask):** The backbone of the application, running in **main.py**. It exposes a REST API that the user interface can communicate with.
- **Manual Override Router:** The central logic in **nl_query_service.py**. Instead of relying on a single method, it first attempts to answer a query with a fast, pattern-based system. If the query is too complex, it intelligently escalates to the more powerful Text-to-SQL agent.
- **Text-to-SQL Agent (The "SQL Architect"):** A **LangChain SQL Agent** powered by a local **Ollama** model (**codellama**). This agent is given a highly specific custom prompt and access to the database's enriched schema (including your descriptive comments), allowing it to write precise PostgreSQL queries for complex questions.
- **Vector Search (The "Researcher"):** Uses a **Hugging Face Sentence Transformer** model to convert the user's query into a vector embedding. It then queries a **Supabase pgvector** database to find and retrieve relevant, unstructured documents for added context.
- **Narrative LLM (The "Reporter"):** A fast, powerful **Groq API** model (**Llama3**). Its sole job is to synthesize the final answer. It receives the structured data from the SQL query and the unstructured context from the vector search and generates a final, credible, human-readable narrative.
- **Database (Supabase PostgreSQL):** Serves as a dual-purpose data store for both the structured, relational vessel data and the unstructured vector embeddings for semantic search.

Data Flow: Anatomy of a Query

The application processes a user's question in a sophisticated, multi-stage pipeline to ensure both accuracy and contextual richness.

- 1 A user submits a natural language question (e.g., "*tell all you know about vessels type brig*") to the `POST /query` API endpoint in **main.py**.
- 2 The server passes the query to the **process_nl_query** function.
- 3 The function first attempts a fast, **rule-based** conversion to SQL.
- 4 The system checks the confidence of the rule-based result. If it's low, the **manual override** invokes the **codellama SQL Agent**.
- 5 The **codellama agent**, guided by its custom prompt and the database comments, generates and executes a precise PostgreSQL query.
- 6 In parallel, the user's query is sent to the **Vector Search** component to find relevant documents from the Supabase vector store.
- 7 The final two pieces of information—the **structured data** from the SQL query and the **unstructured context** from the vector search—are sent to the **Groq Llama3** model.
- 8 The **Narrative LLM** synthesizes these inputs into a final, credible answer.
- 9 A complete JSON object containing the generated SQL, the raw data, and the final narrative is returned to the user's application.

Deployment

The application is self-contained and designed for straightforward deployment.

Hugging Face Spaces

Hugging Face Spaces is an excellent platform for hosting demos.

- 1 **Create a `requirements.txt` file:** From your active virtual environment, run `pip freeze > requirements.txt`. This lists all necessary Python libraries.
- 2 **Rename `main.py` to `app.py`:** Hugging Face Spaces typically looks for a file named `app.py` to run.
- 3 **Upload Files:** Upload your project to a new Space, ensuring your file structure is preserved (e.g., the `app/services/` directory).
- 4 **Use Hugging Face Secrets:** Do **not** upload your `.env` file. Instead, go to your Space's "Settings" page and add your environment variables (like `POSTGRES_USER`, `GROQ_API_KEY`, etc.) as **Secrets**. The application will be able to access them securely.

Cloning for General Deployment

To deploy the application on any server that runs Python:

- 1 **Clone the Repository:** `git clone <your_repository_url>`
- 2 **Set up Environment:** Create a Python virtual environment and install the dependencies:
`pip install -r requirements.txt`.
- 3 **Configure:** Create a `.env` file in the root directory and add the required database and API key credentials.
- 4 **Run:** Start the application with `python app.py`.